GhostWriting: Character-Level Text Generation via Long Short-Term Memory Networks

Project Report

November 25, 2025

Abstract

Abstract: This project, titled "GhostWriting," aims to develop a generative text model capable of producing coherent English text character-by-character. Unlike standard classification tasks, this generative modeling problem requires the system to learn the probabilistic structure of language—including grammar, punctuation, and vocabulary—without explicit rule-based programming. The core method employs a Recurrent Neural Network (RNN) with Long Short-Term Memory (LSTM) architecture. The model was initially trained on the philosophical writings of Friedrich Nietzsche to learn complex sentence structures. Subsequently, Transfer Learning was applied to fine-tune the model on a Shakespearean corpus. Results demonstrate that the model successfully captures stylistic nuances, generating valid English words and sentence structures. Qualitative analysis using temperature sweeping reveals that a temperature setting of 0.6 yields the optimal balance between coherence and creativity. The project concludes that Character-Level LSTMs are highly effective for sequence generation and adaptable to new styles via transfer learning with minimal computational cost.

1 Introduction

Natural Language Processing (NLP) often focuses on classification or translation, but generative modeling represents a unique challenge: teaching a machine to create. The goal of this machine learning project is to build a model that can predict the next character in a sequence given a history of previous characters. By recursively updating the input with the prediction, the model generates entirely new text.

The problem is significant because it touches upon the core of artificial intelligence: understanding context and long-term dependencies. Standard Recurrent Neural Networks (RNNs) often fail at this due to the "vanishing gradient" problem, where the network forgets early information in long sequences. To solve this, we utilize Long Short-Term Memory (LSTM) networks, which possess internal gating mechanisms to regulate information flow. The motivation stems from the desire to explore how well deep learning architectures can mimic distinct human literary styles, specifically the dense prose of Nietzsche and the dramatic dialogue of Shakespeare.

2 Literature Review and Related Work

Text generation has evolved significantly over the decades. Early approaches relied on **N-Gram models**, which calculated the statistical probability of a character or word based strictly on the immediate $N-1$ predecessors. While computationally simple, N-Grams lack the ability to maintain long-term context, resulting in incoherent text over long sequences.

Vanilla Recurrent Neural Networks (RNNs) improved upon this by maintaining a hidden state to capture history. However, as established by Hochreiter and Schmidhuber (1997), standard RNNs suffer from vanishing gradients, making them incapable of learning dependencies separated by large time gaps.

Long Short-Term Memory (LSTM) networks were introduced to address this. LSTMs introduce a "cell state" and three gates (input, forget, output) that allow the network to choose what to remember or forget over long sequences. This project builds upon the established efficacy of LSTMs for sequence generation, distinguishing itself by applying **Transfer Learning**. While modern Transformers (like GPT) are the current state-of-the-art, they are computationally expensive. This project demonstrates that LSTMs remain a viable, efficient solution for style-specific text generation on lighter hardware.

3 Problem Definition and Methods

3.1 Task Definition

Formally, given a sequence of characters $C = \{c_1, c_2, ..., c_t\}$, the task is to predict the character c_{t+1} that maximizes the probability $P(c_{t+1}|c_1, ..., c_t)$. This is treated as a multi-class classification problem at each time step, where the classes are the unique characters in the vocabulary.

3.2 Machine Learning Algorithms

We utilize a **Character-Level LSTM**. The architecture processes inputs as follows:

1. **Vectorization:** Text is converted into integer sequences.
2. **Embedding:** Integers are mapped to dense vectors to capture relationships.
3. **LSTM Processing:** The recurrent layer processes the sequence, maintaining a cell state C_t and hidden state h_t.
4. **Dense Output:** A fully connected layer projects the LSTM output to logits representing the vocabulary size.

3.3 Original Techniques: Transfer Learning

A key innovation in this implementation is the use of Transfer Learning. After the model converges on the complex vocabulary of Nietzsche, the learned weights are frozen/transferred and fine-tuned on a Shakespeare dataset. This allows the model to adapt to a script-writing format (e.g., "ROMEO:", "VOLUMNIA:") much faster than training from scratch.

4 Implementation Details

4.1 Datasets

- **Primary Dataset:** Writings of Friedrich Nietzsche. Approx. 600,000 characters. Chosen for its complex vocabulary and distinct voice.
- **Transfer Dataset:** Works of William Shakespeare. Approx. 1.1 million characters. Chosen to test stylistic adaptation.

4.2 Preprocessing Pipeline

1. **Tokenization:** Text is converted to lowercase and split into characters.
2. **Vectorization:** A mapping dictionary maps unique characters to integer IDs (0-83).
3. **Sequence Generation:** The text is sliced into sequences of length $T = 100$. The target sequence is the input sequence shifted one position to the right.
4. **Batching:** Data is shuffled and batched (Batch Size = 64) for training efficiency.

4.3 Model Architecture

The model was built using TensorFlow/Keras:

- **Embedding Layer:** Input dim = Vocab Size, Output dim = 256.
- **LSTM Layer:** 1024 Units, `return_sequences=True`, `stateful=True` (during inference).
- **Dense Layer:** Units = Vocab Size (84).

4.4 Training Procedure

- **Optimizer:** Adam.
- **Loss Function:** Sparse Categorical Crossentropy (`from_logits=True`).
- **Training Time:** 20 Epochs for the base model; 5 Epochs for the transfer learning phase.

5 Experiments and Results

5.1 Training Performance

The base model was trained for 20 epochs. The loss curve showed a steady decrease, starting at approximately 2.74 and converging to 0.76. This indicates that the model successfully minimized the prediction error and learned the underlying distribution of the Nietzsche corpus.

5.2 Text Generation and Temperature Sampling

We evaluated the model qualitatively by generating text at different "temperatures" (a hyperparameter controlling randomness).

- **Low Temperature (0.2):** The output was highly repetitive and conservative.
- **High Temperature (1.0+):** The output was creative but prone to spelling errors and nonsensical words.
- **Medium Temperature (0.6):** This was identified as the "sweet spot," generating coherent sentences with correct grammar and spelling.

5.3 Transfer Learning Results

After fine-tuning on Shakespeare for only 5 epochs, the model successfully adapted its style.

- **Input Seed:** "ROMEO: "
- **Generated Output:** Included dramatic formatting (names in caps) and archaic terms like "thou," "hath," and "lord," proving the efficacy of the transfer learning approach.

6 Discussion and Analysis

Strengths: The character-level approach is vocabulary-agnostic; it does not require a massive dictionary of words, allowing it to invent believable names or words (neologisms). The LSTM architecture successfully captured long-range dependencies, evidenced by its ability to close quotations and parentheses.

Limitations: As a character-level model, it occasionally generates non-existent words, although they are phonetically plausible. Furthermore, while the grammar within a sentence is often correct, the long-term semantic coherence (meaning across paragraphs) is limited compared to larger Transformer-based models.

Challenges: The primary challenge was training time. RNNs cannot be parallelized as easily as Transformers. Additionally, finding the correct temperature for generation required iterative experimentation.

7 Conclusion and Future Work

This project successfully implemented a generative LSTM capable of mimicking complex literary styles. The experiments confirmed that LSTMs can learn syntax and spelling purely from raw character sequences. Furthermore, we demonstrated that Transfer Learning is a powerful technique for adapting pre-trained generative models to new domains with minimal data and training time.

Future Work:

1. **Word-Level Modeling:** Implementing a word-level LSTM to improve semantic coherence.
2. **Architecture Upgrade:** Replacing the LSTM with a Transformer architecture (e.g., GPT-2) to leverage self-attention mechanisms for better long-term context.
3. **Bidirectional LSTMs:** Utilizing Bidirectional layers to capture context from both past and future characters (though this is more applicable to filling in blanks than pure generation).

References

[1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[2] J. L. Elman, “Finding Structure in Time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.

[3] TensorFlow, “Text generation with an RNN,” *TensorFlow Core Tutorials*. [Online]. Available: https://www.tensorflow.org/text/tutorials/text_generation. [Accessed: Nov. 25, 2025].

[4] A. Karpathy, “Neural Networks: Zero to Hero,” *YouTube*. [Online]. Available: https://www.youtube.com/@AndrejKarpathy.

[5] Sentdex, “Deep Learning with Python, TensorFlow, and Keras,” *YouTube*. [Online]. Available: https://www.youtube.com/sentdex.